

GreenFarm Architecture & Mechanics

Deep-Dive

PART 1: Backend Mechanics (FastAPI & Firestore)

1. `main.py` (API Gateway)

- `root()`: Returns a static JSON dictionary. It serves as a health-check ping to verify the server is active without querying the database.
-
- `app.add_middleware(...)`: Injects CORS headers into every response. By setting `allow_origins=["*"]`, it mechanically instructs the browser/client that requests from any domain (like your Flutter web/mobile client) are permitted to read the JSON responses.
-

2. `firebase_service.py` (Database Connection)

- **Initialization Logic** (if not `firebase_admin.apps`): Checks the internal Firebase app registry. If the array is empty, it uses `credentials.Certificate()` to parse the dictionary from `config.py` and establishes the secure channel to Google's servers. This specific check prevents FastAPI from throwing a "default app already exists" crash during server hot-reloads.
-
- `get_db()`: A simple dependency injection function. It returns the active `firestore.client()` instance so routes don't have to initialize their own connections.
-

3. `auth.py` (Authentication Controller)

- `register_user(payload: RegisterRequest)`:
- - *Normalization*: Runs `payload.email.lower().strip()`. This is critical because Firestore document IDs are case-sensitive; "User@mail.com" and "user@mail.com" would otherwise create duplicate accounts.
 -
 - *Validation*: Calls `user_ref.get().exists` to check if a document with that exact email ID is already in the `users` collection.
 -
 - *Insertion*: Uses `user_ref.set(...)` instead of `.add()`. `.set()` creates a document with a specific ID (the email), whereas `.add()` would generate a random alphanumeric ID.
 -
- `login_user(payload: LoginRequest)`:
- - *Retrieval*: Fetches the user document. If `!exists`, it throws a 404 HTTP Exception.

-
- *Sub-collection Query*: Executes `db.collection("users").document(email).collection("products").stream()`. It iterates through this generator using a list comprehension (`[product.to_dict() for product in ...]`) to instantly bundle all the farmer's previously uploaded items into the login response.
-

4. `cart.py` (Cart State on Server)

- `add_to_cart(payload: CartItemPayload):`
-
-
- *Array Parsing*: Fetches the current cart document. If it exists, it extracts the `items` array; otherwise, it initializes an empty list `[]`.
-
- *Deduplication*: Uses Python's `next(..., None)` against a generator expression to scan the array for an existing `productId`.
-
- *Mutation*: If the item is found, it directly accesses the dictionary key `existing["quantity"]` and increments it. If not found, it uses `.append()` to push the new product dictionary into the list. Finally, it overwrites the entire document using `.set({"items": current_items})`.
-

5. `orders.py` (Checkout Processing)

- `checkout(payload: CheckoutRequest):`
-
-
- *Server-Side Calculation*: Computes the total using `sum(item.price * item.quantity for item in payload.items)`. It does *not* trust a "total" sent from the frontend, preventing users from altering the API request to pay \$0.
-
- *ID Generation*: Uses `datetime.utcnow().strftime('%Y%m%d%H%M%S')` appended to "ORD-" to generate a chronologically sortable string ID (e.g., ORD-202608121530).
-

6. `products.py` (Catalog Controller)

- `serialize_product(doc)`: A utility function. Firestore `to_dict()` does not include the document's ID by default. This function takes the raw document, calls `to_dict()`, and manually injects `data["id"] = doc.id` before returning it to the client.
-
- `create_product(...):`
-

- *ID Construction*: Concatenates the user's email with a millisecond UTC timestamp to guarantee a unique, user-traceable ID.
-
- *Dual-Write Pattern*: Executes two `.set()` operations consecutively. First, it writes to the global `products` collection (for the home screen feed). Second, it writes an identical copy to the `users/{user_id}/products` subcollection (for the user's profile view).
-

PART 2: Frontend Mechanics (Flutter/Dart)

1. `user_session.dart` (The Core Engine)

This is the most mechanically complex file, acting as a local database and state manager.

- **Singleton Pattern** (`factory UserSession() => _instance`): Ensures that every screen accessing `UserSession()` points to the exact same place in memory.
-
- `loadSession()` / `saveSession()`:
-
- *Serialization*: `SharedPreferences` can only save basic data types (Strings, Ints). `_accountToMap` converts complex objects (like `CartItem` classes) into standard Dart `Map<String, dynamic>`. `jsonEncode()` then converts that map into a massive JSON string for disk storage. `loadSession()` reverses this process using `jsonDecode()`.
-
- `_normalizeProductForStorage(product)`:
-
- *Data Sanitization*: Flutter's `File` objects (used for uploaded images) cannot be converted to JSON. This function intercepts the product map, extracts the `File.path` string, assigns it to `imagePath`, and deletes the raw `File` object from the map before it gets saved.
-
- *Key Unification*: Because the mock data and API might use slightly different keys (`name` vs `title`, `image` vs `imageUrl`), this function maps all possibilities into a strict, unified format to prevent null errors elsewhere in the app.
-
- `addToCart(product)`: Uses `indexWhere` to scan the cart array. If it returns `-1` (not found), it adds a new `CartItem`. If it returns `>= 0`, it accesses that index and runs `quantity += 1`. It immediately calls `notifyListeners()` which triggers a re-render on any UI element listening to the session.
-

2. `main_shell_screen.dart` (Navigation State)

- **IndexedStack**: Instead of navigating to new pages (which destroys the previous page's state and scroll position), this widget loads all 4 main screens into memory at once, layering them like a deck of cards. Changing `_selectedIndex` just dictates which card is visible on top.
-
- **ListenableBuilder**: Wraps only the Shopping Cart icon. Instead of rebuilding the whole screen when an item is added to the cart, this specifically listens to `UserSession()`. When `notifyListeners()` fires, only the red notification badge redraws.
-

3. `product_detail_screen.dart` (Interactive Logic)

- **Image Rendering Cascade**: Uses ternary operators to check for the safest image source.
 1. Checks `imageFile != null && imageFile.existsSync()`. If true, uses `Image.file()`.
 - 2.
 3. If false, falls back to `imageUrl.isNotEmpty`. Uses `Image.network()`.
 - 4.
 5. If both fail, renders an `Icon(Icons.image)` fallback box to prevent a crash.
 - 6.
- **Batch Cart Addition**: When the user changes the `_quantity` state variable and clicks "Add to Cart", the button executes a `for` loop, running `UserSession().addToCart(product)` X times consecutively.
-

4. `cart_screen.dart` (Calculations & UI)

- **totalPrice & totalItems**: Utilizes Dart's `.fold()` method. This iterates through the list, carrying a running total (`sum`), adding `item.quantity` or `price * quantity` to the accumulator until the final integer/double is produced.
-

5. `checkout_screen.dart` (Validation Mechanics)

- **_isFormValid getter**: A boolean gatekeeper.
 - It checks basic text fields using `.trim().isEmpty`.
 -
 - If the payment method is "Card", it strips whitespace from the card string using `replaceAll(RegExp(r'\s+'), '')` and checks if the length is `>= 12`.
 -
 - If this getter returns false, `_confirmOrder()` terminates early and triggers a `SnackBar` error, preventing the bad data from reaching the session or backend.
 -

- `_buildQrCodePreview()`: Uses a `GridView.builder` with mathematical logic (`row % 2 == 0`, etc.) to dynamically paint black and white squares on a canvas, creating a fake, scalable QR code pattern purely out of Flutter `Container` widgets.

6. `discover_screen.dart` (Canvas Painting)

- `MapGridPainter`: Overrides Flutter's `CustomPainter`. In the `paint()` method, it uses two `for` loops. The first increments by 70 pixels horizontally, drawing vertical lines (`canvas.drawLine`). The second increments by 45 pixels vertically, drawing horizontal lines. This mechanically generates a grid simulating city streets directly onto the GPU without needing an image file.
- `Positioned Widgets`: Uses absolute coordinates (`top: 20`, `left: 50`) inside a `Stack` to float the location pins arbitrarily over the drawn map background.

7. `profile_screen.dart` (Forms & Scoped State)

- `_showUploadProductSheet()`: Opens a `ModalBottomSheet`. Standard bottom sheets in Flutter cannot update their own UI (like showing a selected image) because they exist outside the main screen's `setState` tree. To fix this, it uses a `StatefulBuilder`. The `_pickImage` function is passed the local `setModalState` function so it can force *just the bottom sheet* to redraw when the user selects a photo.

8. `mock_data.dart` & `product.dart` (Polymorphic Logic)

- **Abstract Classes**: Defines `Product` as an abstract blueprint. It cannot be instantiated directly.
- **Overrides**: Subclasses (`VegetableProduct`, `FruitProduct`) inherit the blueprint but provide their own specific logic for the `getBadgeColor()` method.
- **getByCategory Query**: Uses `.where()` with `.toLowerCase().trim()` to perform case-insensitive, whitespace-ignoring string matching. It checks if the search string is contained within the category string, or vice versa, ensuring highly forgiving search results.